

如何将千亿文件放进一个文件系统，EuroSys'23 CFS 论文背后的故事

1. 引言

本文的主要目的是解读百度沧海·存储团队发表于 EuroSys 2023 的论文《CFS: Scaling Metadata Service for Distributed File System via Pruned Scope of Critical Sections》，论文全文可以在 <https://dl.acm.org/doi/10.1145/3552326.3587443> 下载。

论文披露了百度智能云**文件存储** CFS 的元数据系统的核心设计，对长期困扰文件系统元数据领域的 POSIX 兼容性和**高扩展性**（特别是写扩展性）难以兼顾的问题进行了解答。这是一个大规模分布式文件系统能否扩展到百亿甚至千亿级别文件数，同时保持高性能稳定性的一个关键问题。

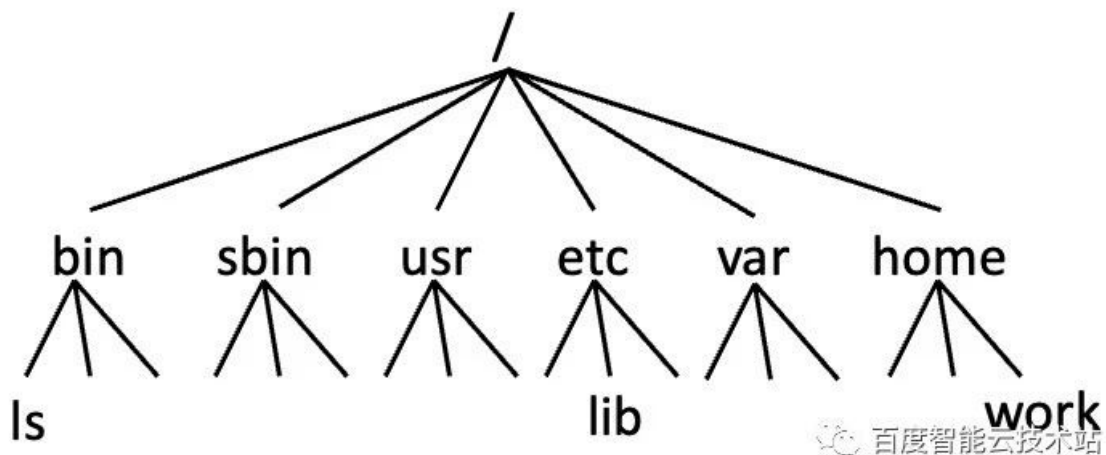
作为一种高度凝练的体裁，论文除了展示必要的分析过程，并不会告知读者这些创新是如何被想到的。我们认为讲清楚这次创新的整个过程，既有助于理解论文本身，也是对论文内容的重要补充。基于这个考虑，我们将整篇文章按照以下结构组织内容：

- 首先，我们概括介绍了文件系统元数据问题产生的背景和业界对该问题的探索，并特别对本文着重解决的写扩展性问题进行了定量分析；
- 然后，我们通过介绍 CFS 元数据架构的演进历史，向读者展示了我们探索的过程以及对元数据问题本质的思考，最终引出论文里介绍的系统架构；
- 最后，我们详细介绍了论文架构的整体设计和其中关键的细节。

2. 背景

2.1. 文件系统的概念

文件系统的定义是一种采用树形结构存储和组织计算机数据的方法，这个树形结构通常被称为**层级命名空间 (Hierarchical Namespace)**。如下图所示，这种命名空间的特点是整个结构像一棵倒挂的树，非叶子结点只能是目录。如果不考虑软链接跟随 (follow symlinks)、硬链接 (hardlink) 的情况，从根节点 (/) 出发，每一个目录项（目录、文件、软链接等合法类型）都可以由一条唯一的路径到达。



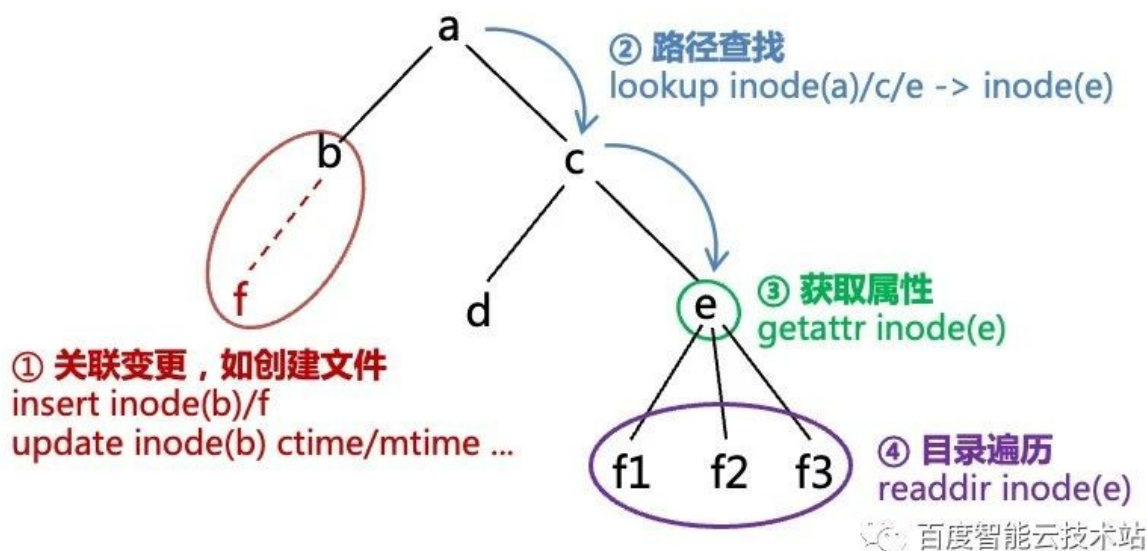
文件系统可以分为元数据 (Metadata) 和数据 (Data) 两个部分。数据部分指的是一个文件具体存储了哪些内容，元数据部分是指层级命名空间树形结构本身。例如，如果我们要读取 /a/b/file 这个文件的数据，元数据部分负责找到 /a/b/file 这个文件存储在哪儿，数据部分则负责把文件的内容读出来。

文件系统主要有两种 POSIX 和 HDFS 两种实现风格：

- POSIX: 全称 Portable Operating System Interface, 是 IEEE 制定的 UNIX 可移植操作系统兼容标准。该标准定义了一个文件系统相关的接口子集, 是文件系统领域最基础和权威的标准。POSIX 兼容文件系统就是指兼容该标准的文件系统;
- HDFS: 源自 Hadoop 大数据生态, 对 POSIX 标准做了一些比较实用的简化和修改, 主要是**放弃**了对 hardlink、随机写的支持, 并增加了一些实用的递归操作。通常也将其归类为 POSIX-like 文件系统, 意思是和 POSIX 相似。

POSIX 和 HDFS 风格的文件系统不存在明显的分界线, 在实现技术上是互通的, 实际使用时通过简单的接口转换也可以互现替换, 当然这种转换会以牺牲一定的兼容性为前提。论文描述的系统是 POSIX 风格的, 但研究成果同样适用于 HDFS 风格的系统, 甚至文章里用于对比的 HopsFS 和 InifiniFS 均是 HDFS 风格的系统。

2.2. 文件系统元数据问题的抽象



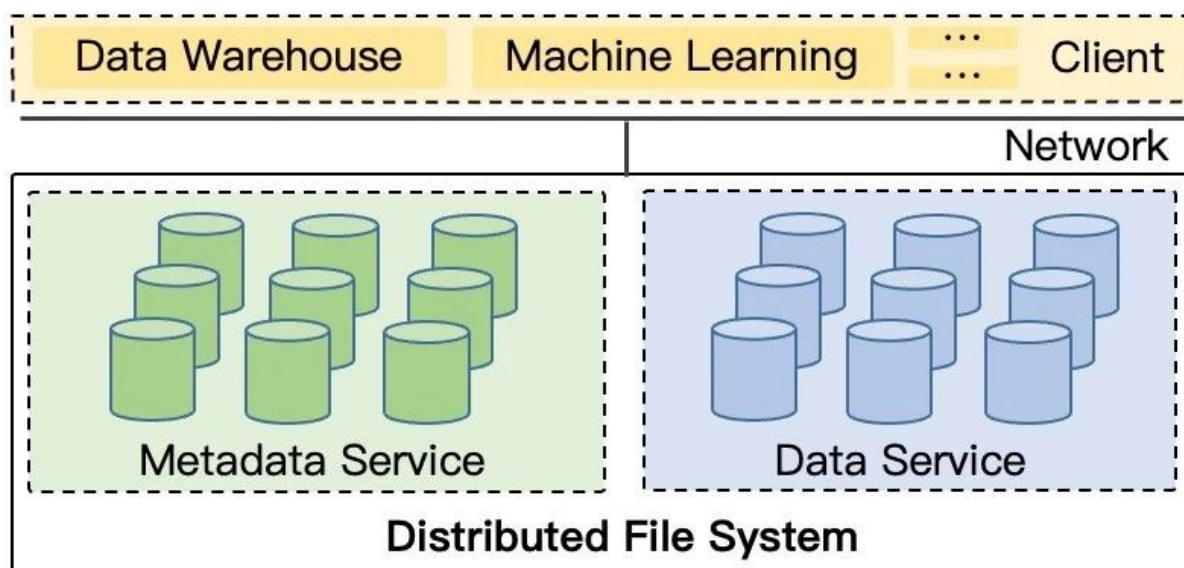
在探索元数据问题的过程中, 我们逐步建立了文件系统元数据问题的抽象, 在此处先进行介绍, 以方便展开后续的内容。一个文件系统的元数据系统只要正确实现了这个抽象, 就解决了 POSIX 兼容性问题的核心部分。

- 写操作的要求
 - **关联变更**: 所有操作均需要实现关联变更 (编号 1), 即增删目录下的子项的同时更新父目录必要的属性信息。关联变更需要原子完成, 即 all-or-nothing, 要么全部都发生变化, 要么全部都不发生变化;
 - rename: rename 的作用是对目录项进行重命名, 这是整个文件系统里最复杂的操作, 除了满足关联变更的要求外, 还有很多其它复杂的要求, 请自行参阅 POSIX 标准的定义。
 - 读操作的要求
 - 点读: 包括路径查找 (lookup, 编号 2) 和 inode 获取属性 (getattr, 编号 3) 两种;
 - 路径查找: 用于一级一级找到对应目录项, 单个 lookup 路径查找操作的输入是父目录 inode + 子项的名称, 返回值是目录项的 inode。inode 是一个文件系统中唯一标记一个目录项的编号, 通常是一个 64 位整数;
 - 获取属性: 获取指定 inode 的属性信息, 属性信息包括 size、nlinks、ctime、mtime 等。
 - 范围读: 目录遍历 (readdir, 编号 4), 获取一个目录下所有子项的列表, 通过一系列的 readdir 调用, 每次返回一部分子项, 直到获取到完全部子项。
- 对于上述的抽象, 我们进一步做如下补充说明:

- 关联变更的本质是准确反映子项列表的变化
 - 一方面，**关系到元数据的一致性**。例如，删除目录的时候需要判断目录是否非空，这在很多系统里会维护一个字段来记录，如果这个字段的信息滞后了，就有误判的可能性。当一个非空目录被误判为空目录，目录本身会被删除，目录下的子项残留了下来。这些残留的子项是所谓的孤儿节点，无法通过路径查找访问到；
 - 另外一方面，**关系到缓存的正确性和效率**。为了优化文件系统元数据的性能，客户端通常会缓存 lookup 和 readdir 操作的结果，然后再次访问的时候可以通过获取父目录的属性信息，快速判断父目录是否被修改，从而确认缓存是否有效。如果没有这个机制，操作要么无条件重新执行保证数据正确，要么容忍可能存在的信息滞后。这个缓存机制对大目录的 readdir 效果尤其重要，可以节省重复执行大目录的 readdir 带来的大量 I/O 开销。
- 读操作隐含的两种索引需求
 - <父目录 inode, 子项名称> 索引：这个索引需要支持点读和范围读，分别对应 lookup 和 readdir。需要特别说明的是，POSIX 对 readdir 并没有规定需要按照字母序返回结果，只需要实现不重复、不遗漏返回即可，这种情况下范围读中的“子项名称”参数可以是某种系统内部的游标 (marker) ；
 - 索引：这个索引只需要支持点读，对应 getattr 请求获取属性信息。

2.3. 分布式文件系统的元数据服务

二十多年前，随着 GFS、HDFS 等分布式文件系统的兴起，**元数据和数据分离**的架构逐渐成为分布式文件系统领域的主流共识。这种架构将整个系统分成负责元数据的元数据服务 (Metadata Service) 和负责数据的数据服务 (Data Service) 两个部分。



百度智能云技术站

对于数据服务，由于不同文件的数据相互独立，同一文件的不同部分也很容易条带化和分块处理，天然具备并行处理的条件，因此较容易扩展到很大的规模。但对于元数据服务，层级命名空间的目录结构带来了很强的父子依赖关系，并行处理并不是件容易的事情。

我们通常通过以下指标来综合衡量一个元数据服务实现的优劣：

- 扩展性：衡量一个实现可以扩展到多大的规模，实际上可以细分为两个指标：
 - 规模扩展性：指的是系统可以存储多少目录项，数亿、百亿，还是千亿。因为文件占比是最高的，通常也将这个指标称为系统支持的文件数；

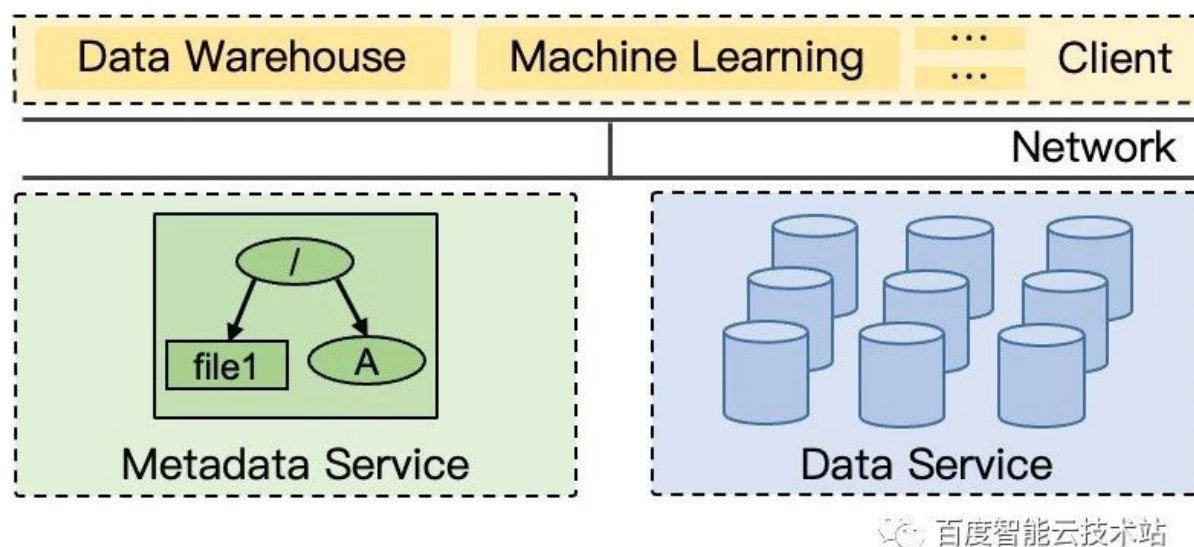
- 性能扩展性：通过增加节点，系统的元数据读写 OPS 能够分别扩展到什么量级，以及读写 OPS 和节点规模是否呈现线性关系。
- 延时：衡量单个请求需要花费多长时间才能完成处理；
- 均衡性：衡量系统是否有能力疏散热点，让不同节点的处理压力大致均衡。

针对扩展性和均衡性，需要特别指出的是，尽管很多实现认为自己具备非常好的扩展性，但是根据木桶原理，扩展性的短板是由表现最差的那个节点决定的。因此，如果一个系统的均衡性没做好，扩展性的实际表现不会特别好。

元数据服务的架构发展在业界经历了三个阶段：

阶段一：单点元数据架构

早期分布式文件系统存储的单个文件都比较大，文件数量不会超过数亿，单点的元数据服务完全可以满足需求。GFS、HDFS 等系统是这个阶段的代表。



阶段二：（耦合式）分布式元数据架构

随着 AI 等新型负载的流行，现代文件系统里存储的文件越来越小，文件数量数却越来越多。单个文件系统的文件大小可能只有数十 KB，文件数却高达数十亿。单点架构的极限是存储和处理数亿文件，已经不能适应时代的需求，因此，可扩展的多节点分布式元数据服务成为必选项。HDFS Federation、Lustre DNE1/DNE2、CephFS、BeeGFS 等系统都是这一阶段发展起来的。

这个阶段的分布式元数据服务是在单点架构上的横向扩展，每个节点同时负责数据存储和文件系统语义的处理。为了和后来的分离式架构做区分，本文将这种架构称为耦合式架构。

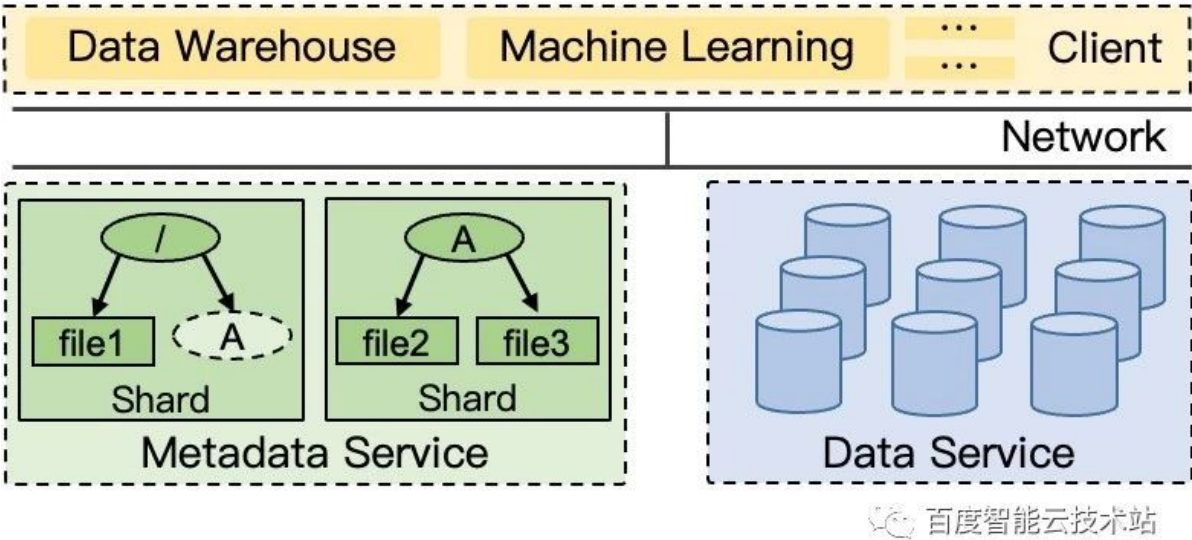
耦合式架构系统普遍采用子树分片或 Hash 分片的方式来将整个层级命名空间分布到不同的节点上，能够大致均匀地在数据量上打散元数据，但是元数据在创建时就确定了其物理位置，**不能动态负载均衡，无法很好的解决数据热点问题。**

尽管有很多研究提出了各种负载动态均衡的方案，某些开源系统代码上也支持该能力，但并没有实际落地生产的案例。归根到底，是因为**元数据架构同时耦合了数据存储和处理逻辑，动态迁移的过程既要完成数据的搬迁，又要保证处理过程不中断或者中断时间极短（数秒钟）**，工程实现的难度极大。

当一个操作需要多个节点参与时，如何保证元数据的一致性是一个比较难的问题，实现上要么需要引入复杂容易出错的多节点参与的分布式锁机制（Lustre DNE2、BeeGFS、CephFS），要么放松对 POSIX 或 HDFS 语义的完整支持（HDFS Federation）。前者在冲突时开销较大使得系统的写扩展性存在瓶颈（后文会有定量分析），后者需要业务改造以感知底层的数据分布。

引入分布式锁的目的是为了正确实现关联变更。关联变更里最核心的一个要求就是原子性。当需要变更的数据分散在多个节点时，分布式锁可以保证变更同时生效或者同时不生效。当然，实现一个能正确容忍各种异常的分布式锁机制也是一个有一定难度的技术活儿，由于和本文关系不大不再展开。

综合来说，这个阶段的系统一定程度上解决了规模扩展性的问题，能够存储几十亿文件，延时指标在请求不跨节点的时候是比较好的，但是写扩展性和均衡性表现不是很好。

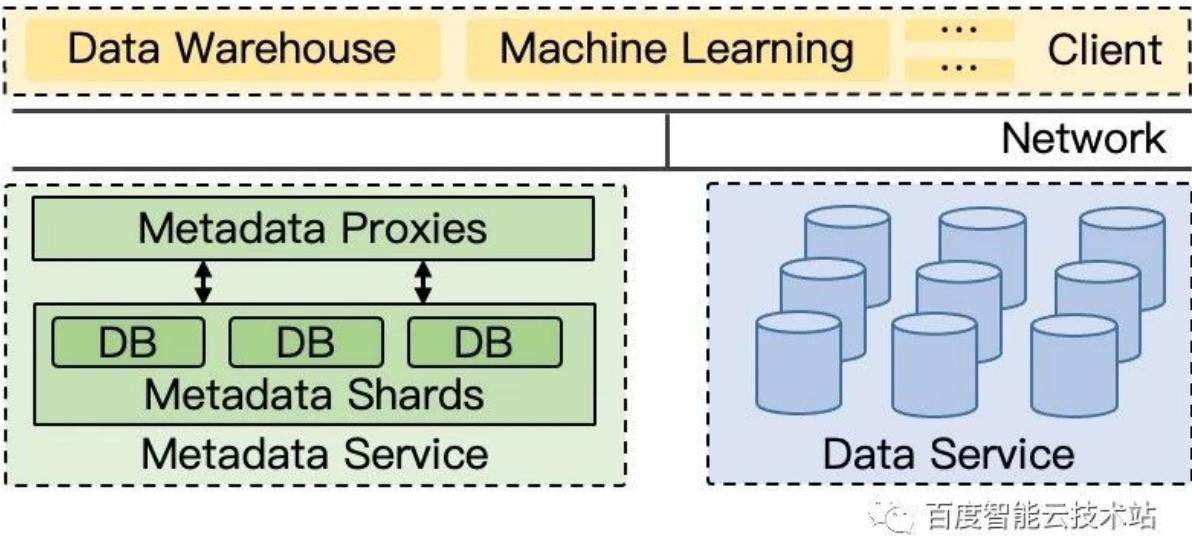


阶段三:分离式元数据架构

这个方法论具体的做法是**将复杂系统进行分层，每一层次专注于解决一个领域的问题做到极致，最后叠加不同层次实现完整的功能**。这在计算机领域是屡试不爽的套路。经典的 OSI 7 层网络模型，近年来在大数据和数据库领域流行的存储计算分离架构，都是最好的佐证。

2017 年，HopsFS、ADLS 两个工作将类似的想法引入到分布式文件系统元数据领域，在这之后出现的重要系统和研究工作几乎都是基于此思路的。该思路将元数据服务从架构上分成两层：

- **元数据代理层**：这一层对外提供 POSIX 或 HDFS 接口，对内将层级命名空间的数据转换成 Table 系统中的记录，处理时利用事务保证操作的正确性。
- **数据库层**：这一层负责数据存储，通常采用 NewSQL 或分布式 KV 系统（以下统称 Table 系统），实现数据的持久化的同时提供分布式事务能力；



分层思路让整个系统的设计变得非常简洁。原来**耦合式架构里同一个节点既负责数据存储还负责文件语义**，两者揉杂在一起剪不断理还乱。分层之后，数据存储的部分首先被从元数据节点里剥离了出去，元数据节点只剩下处理逻辑，变得更专注。

Table 系统提供的 SQL 或者 KV 接口易用性和通用性非常好，和存储交互的逻辑表达起来也很容易。最重要的是，**Table 系统提供了事务**。事务是计算机领域最强大和最有用的能力之一，ACID 配合合适的隔离级别，（几乎）所有的复杂的一致性都可以基于这个能力来简洁地解决，文件系统语义要求的一致性当然也不例外。

除了架构变得更简洁外，分离式架构借助 Table 系统诸多成熟的能力，还解决了之前架构难以解决的规模扩展性和均衡性的问题：

规模扩展性：Table 系统具有非常好的规模扩展性，可以存储海量的数据，例如，在**对象存储**服务上，百度智能云的 TafDB 已经存储了万亿条数据，这个规模还在不断增长。通过将文件系统元数据以合适的格式编码存储到这类系统中，可以满足元数据服务对百亿、千亿文件的存储需求；

均衡性：文件系统的元数据编码到 Table 系统之后，维护层级关系变成了多条记录同时变更时的原子性和一致性问题，这是事务可以保证的。**剥离了文件系统语义之后，数据本身并没有特殊之处，Table 系统可以使用数据分片分裂、合并、均衡机制进行热点的疏散**，这些机制非常平凡和成熟，产生的服务中断时间可以控制在秒级。

分离式架构没有在更早的时间点出现是有其历史原因的。

事务长期只能在单机上良好运作，直到 2012 年 Spanner 横空出世，才让人们明白了怎么去组合利用 Paxos/Raft 这类分布式共识算法、**Percolator 事务机制**来构建一个强大的分布式事务系统。这之后从 Spanner 衍生的各种开源或闭源的 Table 系统就雨后春笋般繁荣了起来，一定规模的技术公司都有能力研发这类系统。分离式元数据架构出现的时间点，刚好吻合这类系统初步成熟并在各类场景开始落地的时间，几乎是水到渠成。

遗憾的是，**分离式架构同样没有解决写扩展性的问题**，写延时的表现甚至比耦合式架构更差：

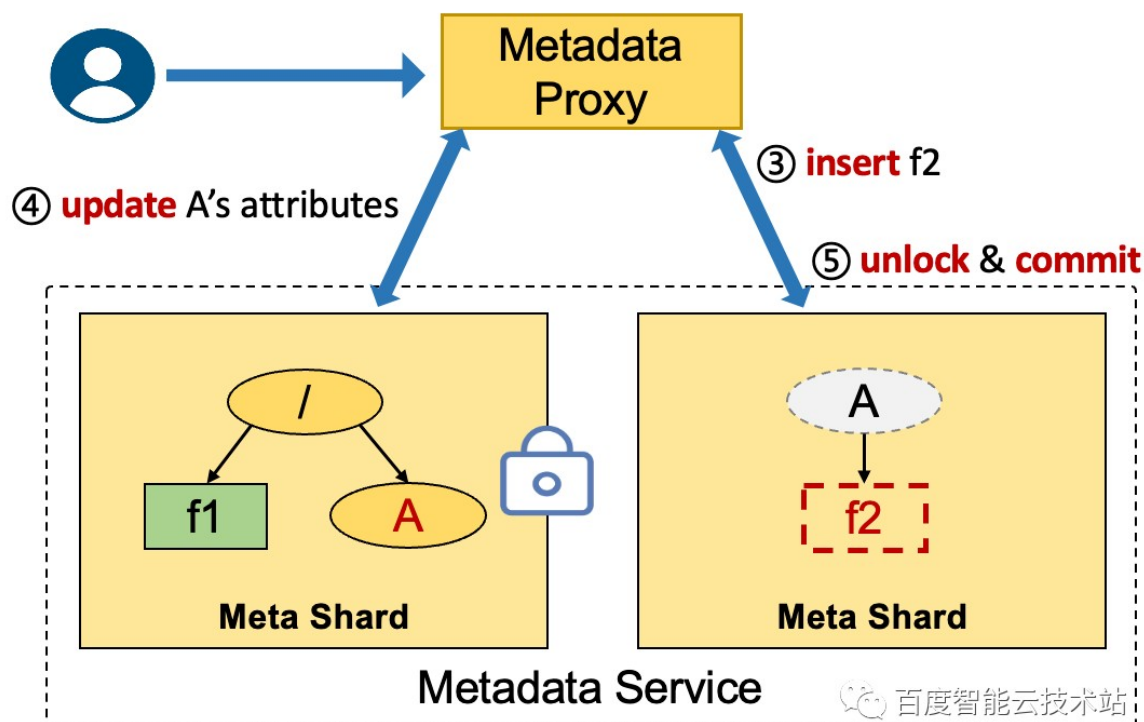
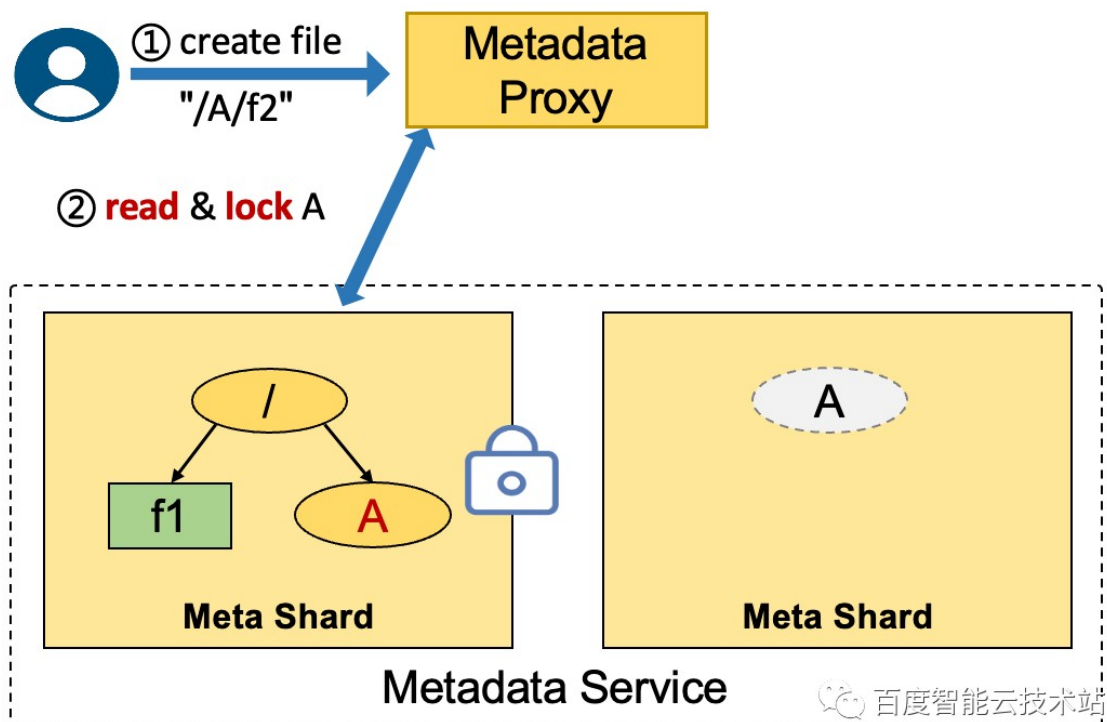
事务在架构里的实质作用其实也是一种分布式锁，并没有解决其它分布式锁机制的缺陷，当一个写操作需要多节点参与时，无论是吞吐还是延时的表现都会比较差；

分离式架构处理请求时，需要先经过元数据代理层，再到数据库层，比耦合式架构的处理路径要更长，因此天然在性能上，特别是延时指标上更具劣势。读请求可以通过客户端直接读数据库层的方法来进行优化，但写请求就没有办法这么处理了。

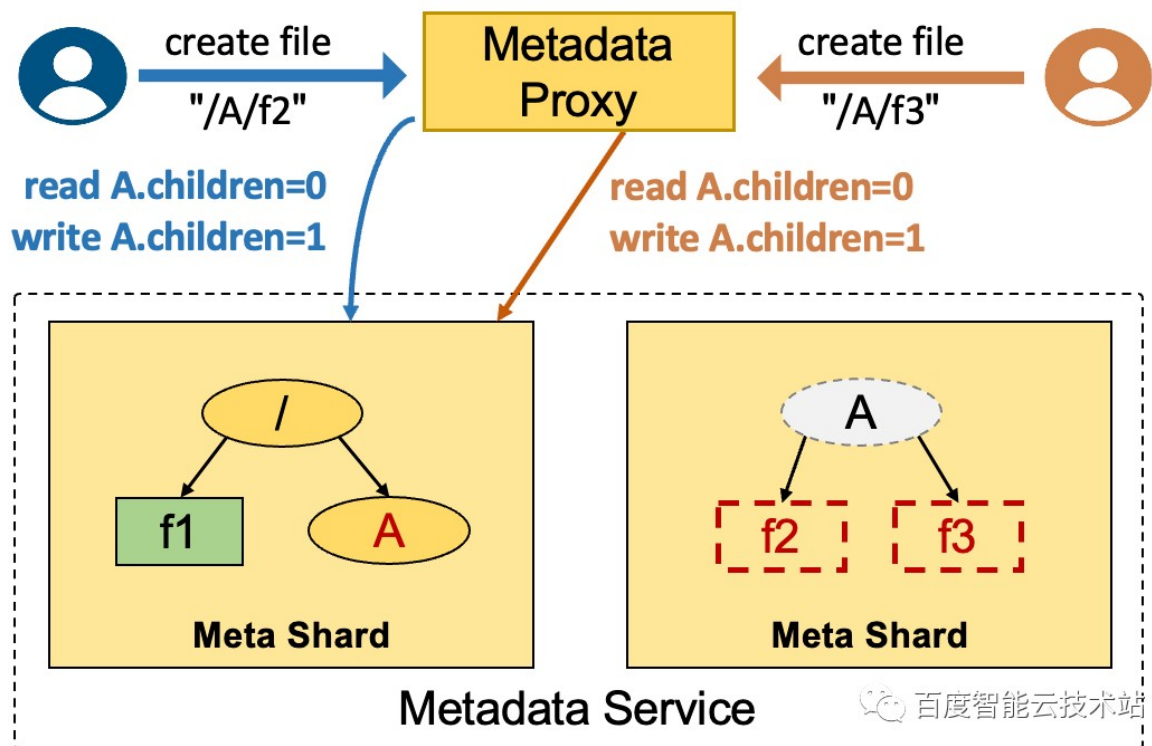
2.4. 为什么元数据服务需要分布式锁

前文我们一直在强调通过分布式锁来保证一致性，这个一致性就是指关联变更的正确性。在这一章节，为了让大家更好地理解这个问题，我们再补充说明一下分布式锁是怎么实现这个保证的。以分离式架构创建文件 create file “/A/f2” 为例，需要经过以下步骤：

1. 创建操作发送到元数据代理；
2. 读取 A 目录的属性并对 A 加锁，在这一阶段实际上还会去检查目录是否存在、用户是否有权限创建等；
3. 插入 f2 文件的记录；
4. 更新 A 目录的属性，主要是 mtime/ctime/links/size 等属性。这个阶段变更还没有提交，客户端不可见；
5. 解锁并提交这次变更，这次提交如果涉及到多个元数据分片，需要采用 **2PC 提交**（two phase-commit）来保证变更同时生效；



操作 2 - 5 需要在锁的保护下进行，如果没有锁的保护，会很容易出现并发问题。如下图所示，系统使用 children 字段表示该目录下有多少个有效的子目录或文件，这个字段可以在删除目录的时候快速判断目录是否为空。当没有锁保护的时候，并发的操作 create file "/A/f2" 和 create file "/A/f3" 分别读到 A.children=0，各自 +1 并更新，最后得到的 children 字段数据是错误的 1。



这个错误会导致后续删除目录操作误判目录为空，使一些文件成为孤儿节点。例如，在这个例子结束之后，用户先执行 delete file “/A/f2”，再执行 delete directory “/A”，这时看到 children=0 认为目录是空的，就可以把 A 目录删除掉了。此时，f3 文件还存在，但却再也无法通过 /A/f3 路径访问到了！

实际系统的并发情况远比上述的例子要复杂得多，但从上述的例子，我们可以简单了解到为什么分布式文件系统的元数据操作需要分布式锁来保护。

2.5. 对分布式锁性能影响的定量分析

在回答了为什么元数据服务需要分布式锁之后，我们在本章节进一步展示分布式锁对写性能的影响。这里直接引用论文里对 HopsFS 创建文件操作做的定量分析，分析结果同样适用于其它系统。

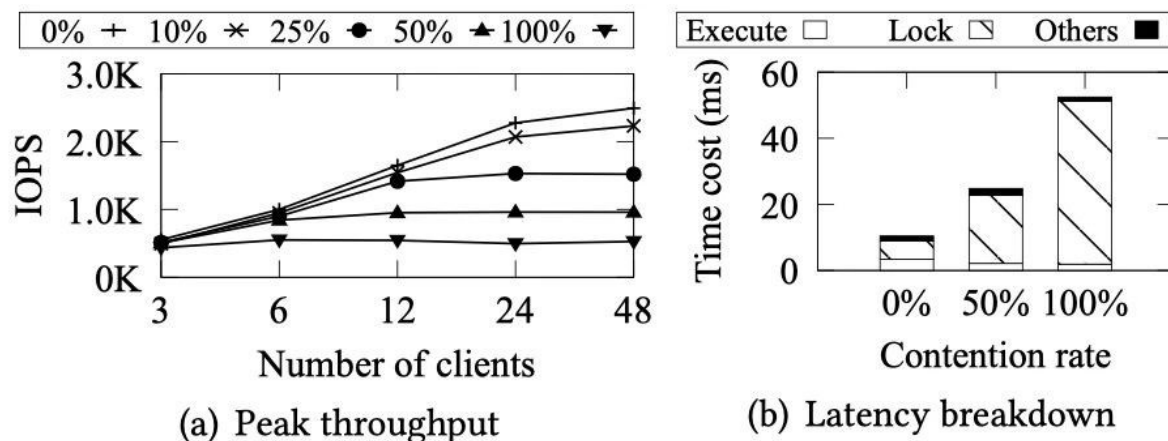


Figure 4. Performance of create within HopsFS under different workload intensity levels and contention rates

在 Peak throughput 这个图里，我们构造了锁冲突比例从 0% 到 100% 的负载，观察系统的 OPS 和客户端数量的关系。在没有冲突的情况下（冲突比例 0%），OPS 几乎随着客户端数量线性增长，但是当冲突比例越来越高时，OPS 有显著的下降，当冲突比例达到 100% 时，整个曲线变得很平，说明系统的性能已经完全丧失了扩展性。

在 Latency breakdown 这张图里，我们进一步分解了 HopsFS 的执行时间，可以发现，当冲突比例为 50% 和 100% 的时候，锁冲突在整个操作中的占比高达 83.18% 和 93.86%。

上述的实验表明，**锁冲突是影响系统扩展性和操作延时的关键。**

另外，我们可以看到一个有意思的结果，那就是即使在无冲突的情况下，锁在整个操作里的耗时占比也达到了 52.9%。这是因为系统并不知道何时会发生冲突，因此需要始终以最坏的情况来预防。如果能降低甚至消除这部分延时，就可以极大的降低系统的延时。事实上，无论是耦合式架构，还是分离式架构，很多系统都会通过一些数据的放置策略，**让一个子树或者一个目录的数据尽可能分布到一个分片上**，以方便做锁冲突的优化。

3. CFS 元数据架构的演进历史

论文披露的 CFS 元数据架构在内部的代号是 Namespace 2.0，即第二代架构。第二代架构对第一代架构存在明显的继承关系，正是因为我们分析清楚了第一代架构的局限性，才使得第二代架构的设计成为可能。

在正式介绍 Namespace 2.0 的架构之前，让我们先花一些时间讲一讲 CFS 元数据服务是怎么一步步演进到今天这个架构的，这个对理解整篇论文是有帮助的。

3.1. Namespace 1.0: 分离式架构上的微创新

当 2017 年我们开始设计 CFS 时，最重要的一个设计目标就是系统应该具备支撑海量文件的能力，这个海量的量级要远超传统系统的量级。云上的文件系统是多租户的，租户数量叠加单个租户的规模，就会导致整个集群的文件数量爆炸式增长。例如，一个用户 10 亿文件，100 个租户就是 1000 亿文件。即使一开始的数据规模没有那么大，但作为面向未来三五年甚至更久的设计，必须具备一定的前瞻性，为未来架构上的扩展留足余地。

我们一开始就将单点架构排除在外。面对大量的租户，单点架构会导致需要非常多的小集群，带来巨大的运维复杂度。另外，随着单个文件系统规模的扩大，单机的能力上限、热点数据这些问题迟早会遇到。这些因素会导致单点架构无法走很远，那就不如在一开始就朝着真正的分布式的方向努力。

在调研的时候，我们注意到 HopsFS、ADLS 这两篇论文的工作，经过研判之后认为这两个工作代表的分离式元数据架构是未来整个领域的发展趋势。当时我们已经看到这个架构在写延时和写扩展性方面的劣势，但我们认为这不过是一朵乌云，汽车刚发明出来的时候还跑不过马车呢，驱散这朵乌云不过是时间问题。

在百度智能云内部，后来以“沧海”品牌命名的新一代存储体系当时正在建设中。这套体系以 **brpc + braft 为基石**，通过组件化的方式实现各类易运维、高性能、超大规模的[分布式存储](#)服务。这个体系的元数据底座 TafDB 是一个类 Spanner 的系统，几乎和 CFS 同时启动研发。

综合这些因素，由 **TafDB 提供海量数据存储能力和分布式事务，CFS 自己实现文件语义层**这条技术路线在原理上确定了下来。在 TafDB 就绪之前，CFS 基于 MySQL 开始前期的研发工作。

确定技术路线之后，我们针对 CFS 的目标业务场景，做了一些设计上的调整：

1. 文件属性分离

这个调整是说将文件属性（file attributes）从元数据服务中剥离出来，和文件数据（file data）一起放到数据服务（Data Service）中进行处理。主要的考虑和性能有关系：

读性能的考虑：在 2.2 章节我们指出，属性部分只需要满足 inode 索引的点读即可，TafDB 作为一种全局有序的存储系统，点读的处理性能要比全局无序的数据服务要低；

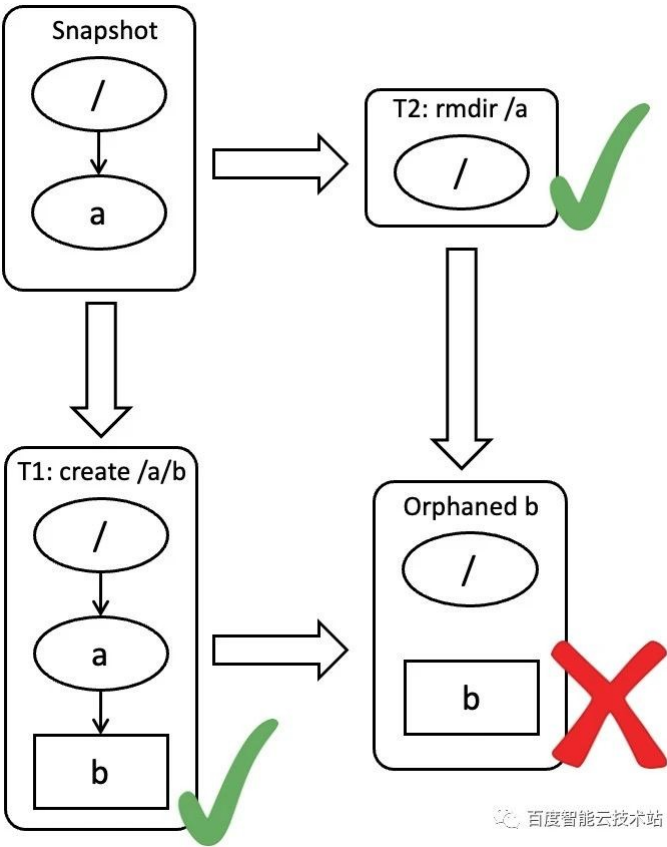
写性能的考虑：在修改文件数据的时候，POSIX 要求修改 ctime、mtime 等属性，追加写还伴随着文件大小（file size）的更新，这会在数据写数据路径上引入频繁的元数据更新操作。**类 Spanner 系统的写 OPS 上限大约为百万级别**，不可能都消耗在文件数据修改操作上，因此，整个系统的写性能将比这个数字更低，不能满足业务的需求。

这个调整让整个系统的文件属性操作的扩展能力变得非常好，后来 Namespace 2.0 沿用了该设计。读者可以在论文实验部分可以找到对该设计收益的定量分析。

2. 读写分离

另外一个调整是将元数据读写路径做了分开处理。对于读请求，我们绕过了元数据代理层直接访问 TafDB，这样可以缩短读延时，同时少了元数据代理层的转发开销之后 OPS 也有一定提升。对于写请求，则**将每个文件系统（CFS 支持多租户，一套系统中存在很多文件系统实例）的写请求收敛到一个单点进行处理**，这么做的原因详述如下。

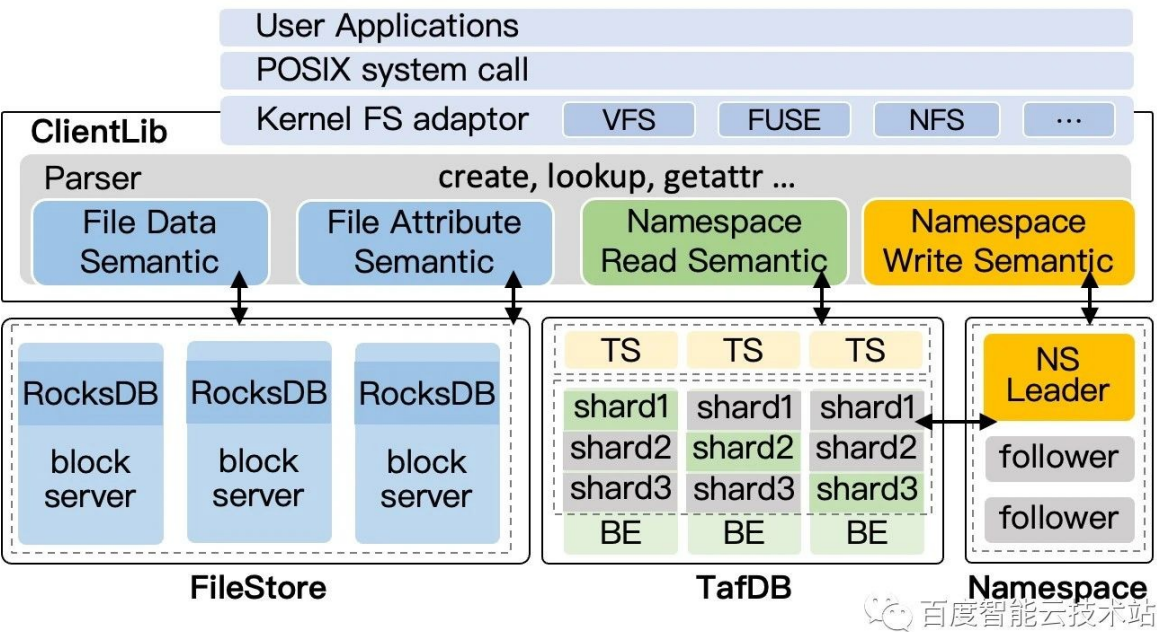
TafDB 提供的隔离级别是快照隔离级别（Snapshot Isolation），但文件系统场景实际需要串行化快照隔离级别（Serializable Snapshot Isolation）来避免孤儿节点问题。在下图的例子中，rmdir “/a” 和 create “/a/b” 操作并发了，在操作开始后，他们读到的是事务开始前的快照数据，同时看到 /a 目录存在，然后都操作成功了，这就导致 b 成为一个不可达的孤儿节点。



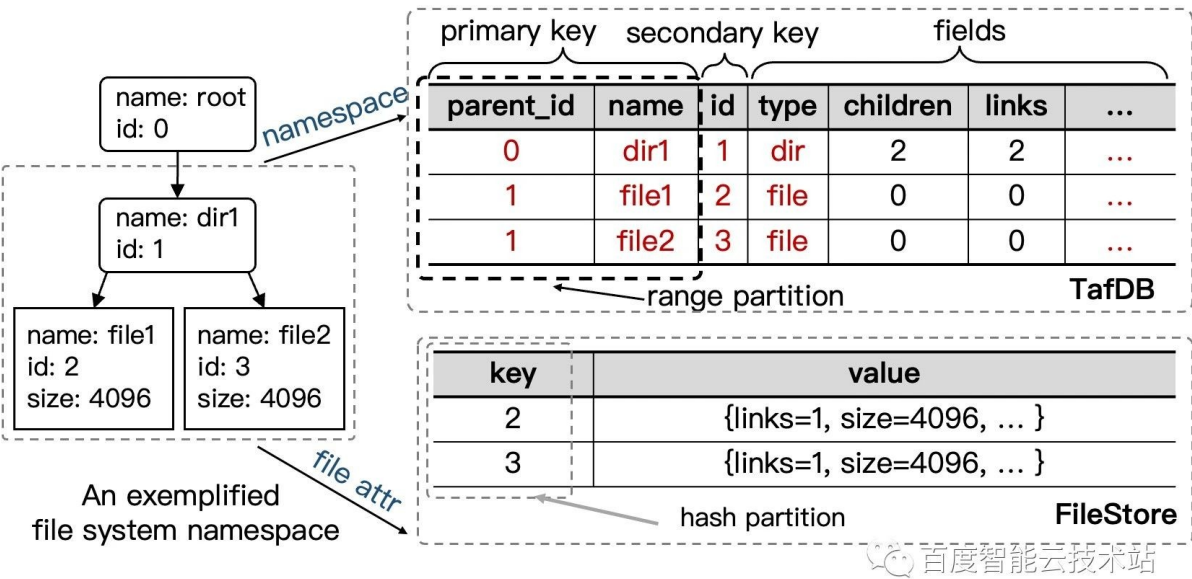
通过在父目录的记录上制造写冲突可以规避该问题，代价是让事务的冲突变得更频繁。我们前文对锁的代价进行过定量分析，这在 TafDB 这种采用乐观锁模型的系统代价更为高昂，因为这类系统判断事务冲突是在提交前的最后一刻，在这之前因冲突回退的事务已经完成了绝大部分的操作，包括多轮 RPC 和落盘开销。

这些代价对于写性能的影响极大，容易导致大量不可控的长尾甚至整个系统雪崩。这个问题在短期内没有特别好的解决思路。同时，根据公司内的经验，单点架构的写性能如果优化好可以达到 5 万 OPS 左右，我们认为这个性能短期内是够用的。因此，我们决定**将每个文件系统的写请求收缩到一个单点进行处理**，写扩展性和写延时的问题留到以后再解决。

经过上面的调整之后，Namespace 1.0 顺利的完成了研发并落地，大概的架构如下图所示。在这个架构里，所有文件相关的操作均由分布式的 FileStore 负责，其它的元数据读操作直接由客户端 ClientLib 发给 TafDB 处理，写操作经过 Namespace 进行处理，Namespace 是一个 Multi-Raft 的实现，每个 Raft 复制组负责一个文件系统。



系统的数据结构如下图所示。其中 TafDB 中的数据存储在一张大表中，以 <parent_inode, name> 为 primary key，负责实现 lookup 和 readdir，以 为 secondary key，负责实现目录的 getattr。



3.2. Namespace 1.X: 1.0 基础上不太成功的探索

1.0 上线之后，CFS 顺利承接了一些元数据读性能要求较高的业务。以一个内部业务为例，迁移到 CFS 之前采用了某开源解决方案，高峰期数十万 getattr OPS 导致元数据节点 CPU 打满，请求大量长尾甚至失败，系统风雨飘摇，徘徊在崩溃的边缘。CFS 很轻松地接下了该业务，整个过程波澜不惊。后来我们分析这个案例，认为主要的收益来自**文件属性的分离**，这个优化使得文件的 getattr 直接由 FileStore 处理，将 getattr 打散得非常充分。

但是写的性能始终是一个痛点，我们基于 1.0 的架构做了大量的实验和分析，尝试提升单机的性能。在这个过程中比较成功的优化包括：

处理流程精简：我们分析了每一类写请求的全流程，从内核到 TafDB，将一些重复的条件检查进行了精简，将可以合并的操作进行了合并；

引入缓存：由于每个文件系统的元数据写都是单点，Namespace 模块任何时刻看到的数据都是最新的，具备引入缓存的条件，缓存命中情况下事务中的大部分读请求都可以被优化掉；

事务冲突预处理：在 TafDB 中出现事务冲突的代价比较大，我们将事务冲突的处理上提到 CFS 这一层，Namespace 模块在执行事务前分析其中存在的冲突点，按照每个冲突点进行排队，从而让下发到 TafDB 的请求无事务冲突。

通过这些优化，我们让写延时和 OPS 分别提升了一倍多。但**和单点架构、耦合式架构相比，这些优化不能抵消处理路径变长带来的开销，写性能指标上仍然存在不小的差距。**另外，如果我们想要将处理能力扩展到多个节点，原来所做的缓存、事务冲突预处理机制在一定程度上会失效，处理延时和单机 OPS 将再度恶化。

到 2019 年年中的时候，事情变得有点儿让人绝望，我们失望的发现，在几乎穷举了所有可能和不可能方案和思路后，摆在我们面前的似乎只有一条路线可走，即回到耦合式架构，基于 TafDB 做二次开发，在尽可能保留 TafDB 的优点的前提下实现文件系统语义。

我们曾经笃信分离式架构是未来发展的大趋势，尽管存在一些显而易见的问题，但汽车刚出来的时候也跑不过马车，解决这些问题不过是迟早的事情。然而，至少在 POSIX 领域，看起来这辆新的汽车终究不是汽车，不过是更好一点儿的自行车。

3.3. Namespace 2.0: 柳暗花明又一村

不管我们怎么看好分离式架构，现实的问题终归还是要解决的。CFS 和 TafDB 两个团队决定最后坐下来一起再努力一把，要是不行就真散伙儿了。

在数学和物理史上，很多新的理论都是从拆除旧理论的基石开始的，典型的例子有非欧几何、相对论。当然，和大师们相比，我们的工作微不足道，这里只是借用比喻一下。

我们决定放下所有的先验知识和成见，去寻找整个文件系统元数据大厦最底层的砖头，从那些砖头开始讨论。

很快，我们发现现有的几乎所有工作，都是从如何满足 POSIX 标准的那一堆接口开始的，但这些接口本身也是有内部结构和逻辑的，还不是最底层的砖头。

通过提炼 POSIX 接口背后的本质要求，我们建立了 2.2 章节描述的抽象模型，并将每种操作作用极其简单的伪代码写了出来。

从这个抽象模型出发，我们推导出一组极其简单的结论：

影响元数据服务扩展性的根源是更新父目录属性时产生的冲突，这是关联变更的一部分；

这些变更本质上只是一些数学运算，分为两种。一种是**加减运算**，针对 links、children、size 等属性（如前文，children 不是标准属性，是为了便于维护目录下的子项数量），每次创建删除子项都需要准确更新。另外一种**赋值操作**，针对 ctime、atime、mtime 等属性，每次创建删除子项的时候用新值覆盖掉旧值；

以上两条对所有修改操作均成立，rename 的要求会更复杂的额外要求。

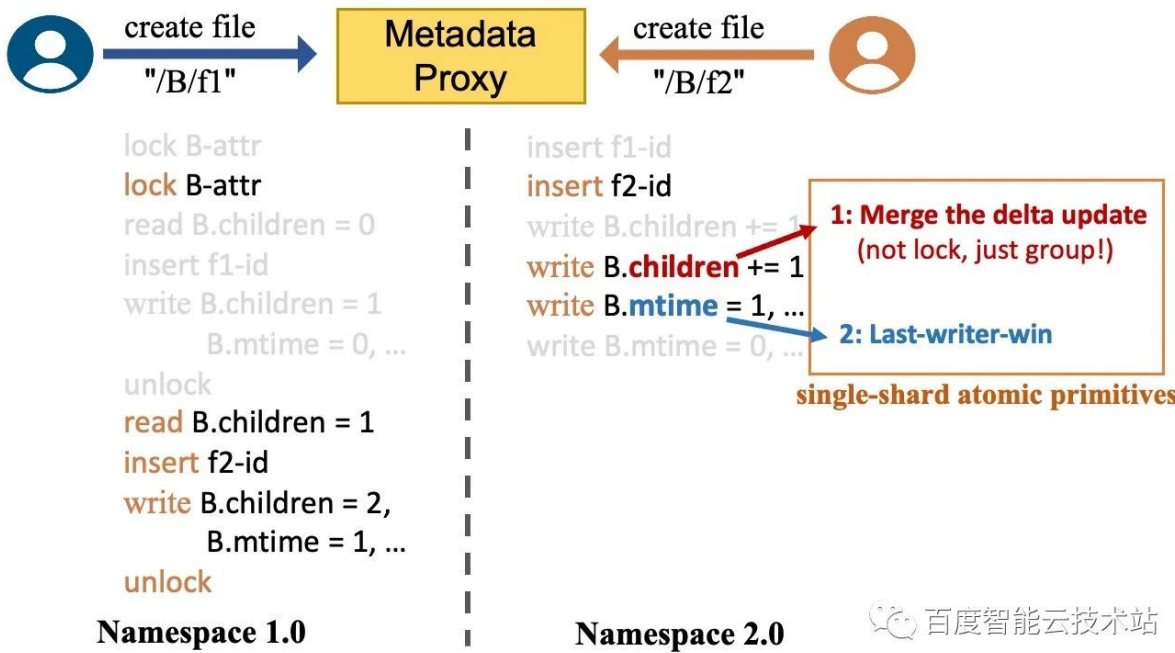
有一定编程经验的同学应该可以发现第二条的抽象要求可以用原子变量操作来表达！传统的实现为了这几个原子变量操作的正确性，将冲突范围至少扩大到整个父目录记录，这是很不精细的做法，类似于给整个操作上了 mutex 锁来保护临界区（critical section）。

在编程上优化这类的问题的路线，就是尝试缩小临界区范围，采用开销更小的保护方式来替代 mutex，如果能优化到仅仅依赖原子变量操作就能保证正确性就最好了。这实际上就是我们接下来采用的优化思路，论文标题对此有比较精准的概括。

计算机领域经常会看到一些知识或经验逐渐变成大家的常识，但当时这些知识或经验产生的过程却是经过一些波折的。

我们总共花了一个季度来讨论和设计 Namespace 2.0，上面这几个关键点现在看来已经成为我们的常识，但在当时耗费了我们一个多月的时间来推导。方案设计完成之后，接下来的工作异常顺利。在已有系统的基础上，我们又花了一个季度做了 demo 版本进行方案验证和性能评估。demo 版本其实和后来的正式版本差别很小，这使得正式开发和测试的周期比较短，到 2020 年 Q1 末的时候 Namespace 2.0 就开始在线上小流量了。此后陆陆续续上线了一些优化，但整体架构直到现在都没有大的变化。

4. 实现思路



Namespace 2.0 的指导思路是不断缩小写操作的临界区范围并最终实现无锁化。上图给了一个简单的示意图，在 Namespace 1.0 中通过锁保护的关联变更，在 Namespace 2.0 仅通过原子操作就满足，并发的操作不再需要串行执行。

为了实现这一点，我们采用了一系列技术的组合：

第一步，通过合适的数据布局，**将冲突范围缩小到单个分片**

系统的首要目标是将 TafDB 的元数据请求的执行范围从跨分片降低到单个分片，这样才可能让去除分布式锁成为可能，否则会有一致性问题，这个在背景部分我们已经论证过。这个目标督促我们重新思考数据布局。

首先，**Namespace 1.0 将文件属性由文件存储单独负责**的做法，已经被验证能够提供更好的文件属性操作性能，Namespace 2.0 继续沿用此设计。

其次，我们观察到，任何一个目录项的元数据需要同时满足两类索引的需求，天然就是两个部分，一部分用于 `getattr`，保存的是属性信息，另外一部分用于 `lookup` 和 `readdir`，保存的是和父目录有关的索引信息。后者是关联变更的一部分，关联变更的剩余部分和父目录属性有关系。通过调整数据布局，将整个关联变更涉及的数据耦合到一个分片上，可以起到让事务冲突聚焦到单个分片的效果。这个调整其实意味着“属性分开存储”这一规则扩展到了包括文件在内的所有类型的目录项。

最后，我们将原来事务保护的整个操作，拆解成两个 TafDB 单分片操作的组合（目录的情况），或一个 FileStore 操作 + 一个 TafDB 单分片操作的组合（文件的情况），配合通过精心排列的顺序，使得这两个操作只需要满足分别满足原子性即可保证执行效果，不再需要一把大锁来保护整个范围。

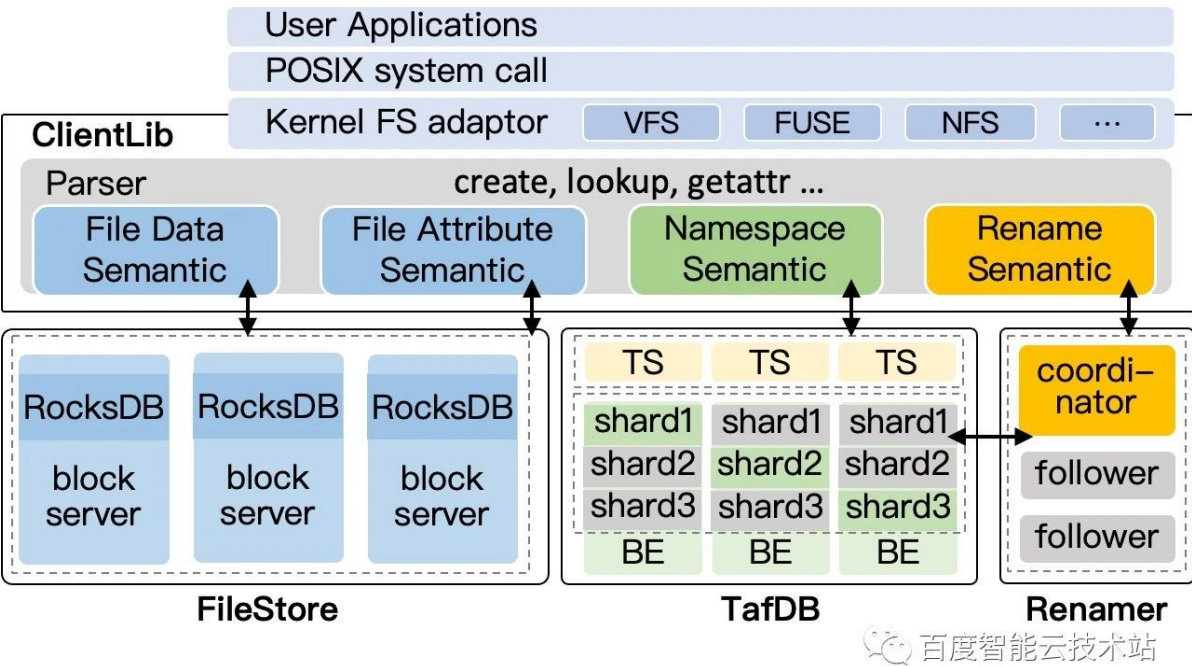
第二步，**将单分片操作的冲突范围缩小到字段级别，并实现无锁化**

在冲突缩小到单分片之后，如果不经任何优化，单个目录下的操作仍然需要串行执行。TafDB 通过对存储引擎进行扩展，引入单分片原语（single-shard atomic primitive）的技术，将原来的行冲突缩小成具体字段的原子操作，并能对并发的操作自动合并。

第三步，精简元数据代理层，进一步缩短执行路径

在上述优化后，除了非常复杂的 rename 操作，元数据代理层（Namespace 1.0 的 Namespace 模块）的作用仅剩实现 POSIX 接口并转发请求，我们将这一层的能力直接整合到客户端中，只保留复杂 rename 的处理，重新命名为 Renamer。

5. 整体架构



根据上述的实现思路，整个系统架构上分成四个部分：

Namespace 存储层 (TafDB)： TafDB 负责存储层级命名空间里除文件属性外的其它部分；

文件存储层 (FileStore)： 这一层是一个以平坦方式组织的块存储层，块是文件数据的基本单位。文件属性和文件数据一起存储在该系统里。**对于文件属性而言，这是一个不支持范围读、只支持点读的 KV 系统；**

Rename 服务 (Renamer)： Multi-Raft 架构，每个文件系统由一个 Raft 复制组提供对复杂 rename，即所谓 Normal Path rename 的支持；

客户端库 (ClientLib)： 客户端负责接收具体的请求，拆解成上述模块的内部请求。ClientLib 提供必要的接口，以方便接入 Linux VFS 层，目前支持 FUSE、Samba、NFS-Ganesha。

从上面的架构图可以看出，CFS 中不再存在传统分布式文件系统的 MDS 或 NameNode 角色，所有的功能全部打散到其它模块。

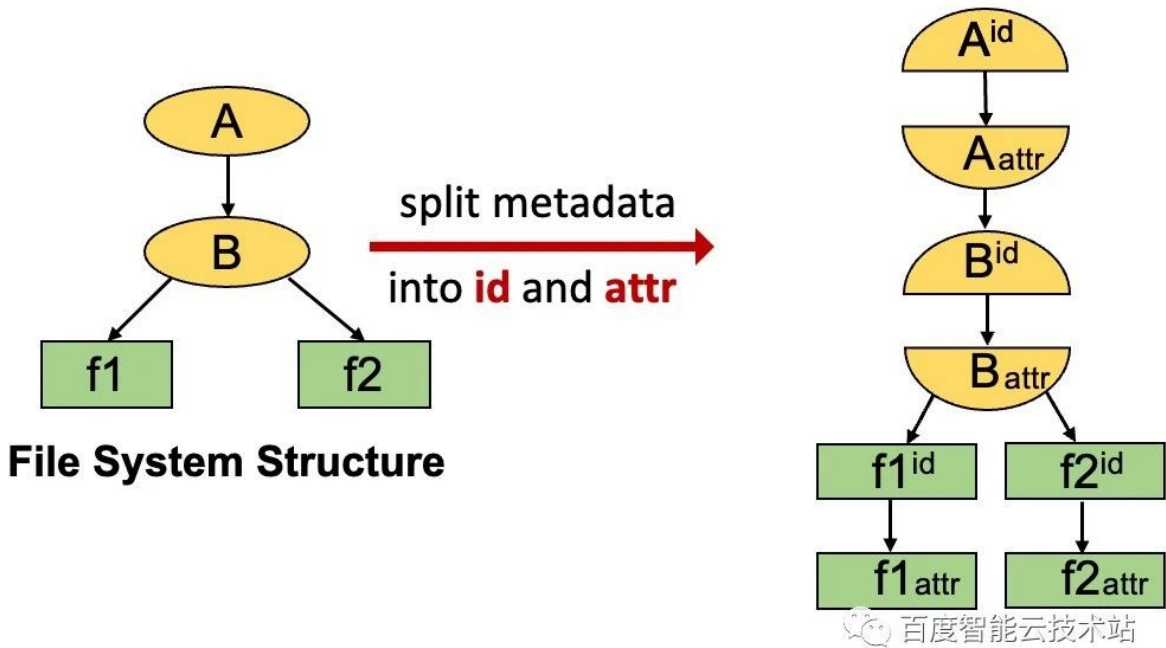
这个设计在维持分离式架构优点的同时，解决了其存在的全部缺点，在扩展性、延时、均衡性等方面都达到了比较好的状态。

我们认为，这个新的思路给分离式架构这辆汽车换上了新的引擎，让这辆汽车在各方面的表现全面超越了马车。

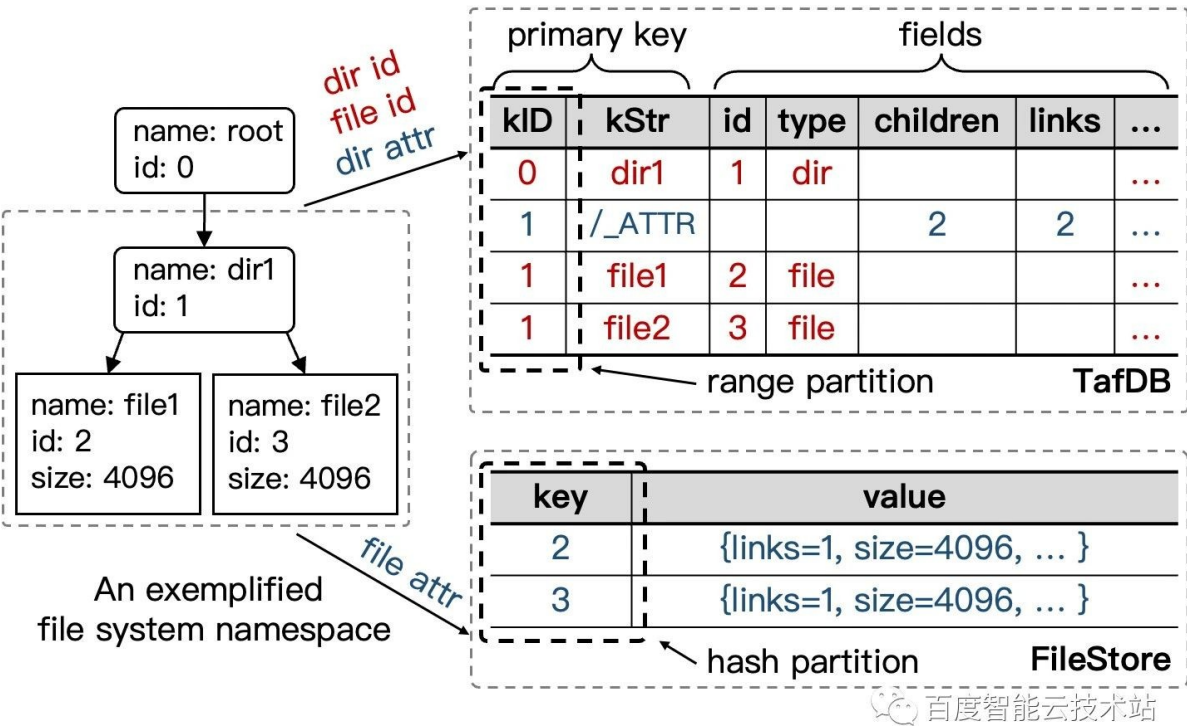
6. 实现细节

6.1. 元数据组织和分片

和 InfiniFS 类似，CFS 将每一个目录项的记录拆解成两部分，分别是 inode id record 和 attributes record。这个分解是进一步缩减冲突域的基础。



TafDB 使用一张 inode_table 表来存储所有的数据，表的主键为 $\langle kID, kStr \rangle$ 。这张表里实际上存储了混合存储的两类数据，包括所有的 inode id record 和目录的 attributes record。FileStore 负责存储文件的 attributes record。整体如下图一样组织：



对于 inode id record, $\langle kID, kStr \rangle$ 中的 kID 部分代表父目录的 inode, $kStr$ 代表子项的名字。这种记录是为了满足 lookup 和 readdir 的需求, 除了 inode 和 type, 其它字段都是无效的。

对于目录的 attributes record, kID 就是目录自己的 inode, $kStr$ 是保留字符串 `/_ATTR`。其它字段存放的是各个属性字段, inode 字段在这里留空没有实际作用。

inode_table 整体上按 <kID, kStr> 有序存储所有数据，分片规则是按照 kID 来的。TafDB 做了一个特殊的保证，即分片无论如何分裂和合并，同一个 kID 的数据始终存储在同一个分片上。这意味着同一个目录的关联变更涉及到的数据都在一个分片进行处理，CFS 实现了目录级别的 range partition。所有分片的分裂和合并操作都是 TafDB 自动进行的，不需要 CFS 关注和人工干预。

TafDB如何做到目录级别的range partition呢？

需要指出的是，我们认为目前的设计没有必要继续探索单目录多分片，以往的耦合式系统里存在此类设计的根本原因是因为一个节点上的热点无法精确疏散，热点会影响同节点上的其它目录的操作，多个热点间也会互相影响，只能通过分散目录压力的方式来缓解问题。但我们的架构不存在此问题，理由如下：

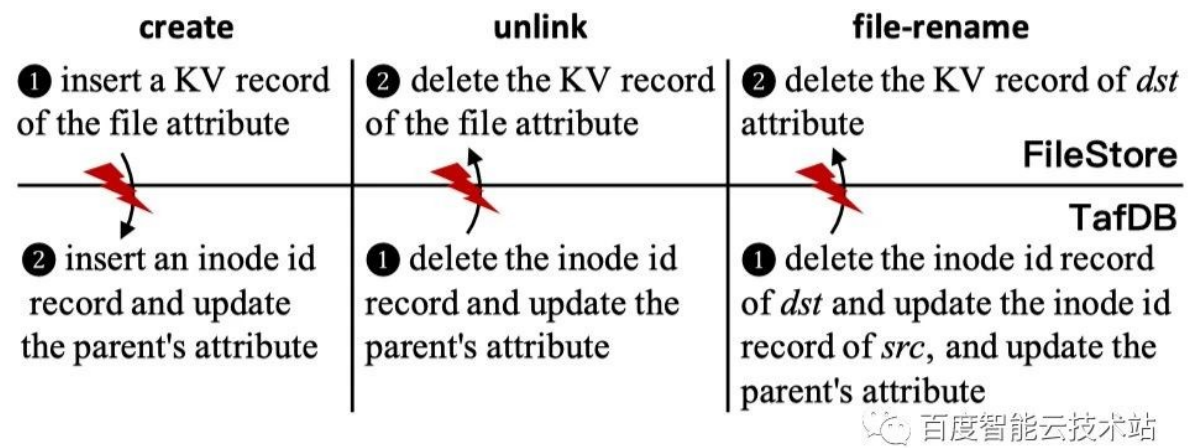
TafDB 极端情况下可以分裂到单个目录独占整个分片，单个分片独占整个机器，单目录的处理能力和单点架构相当。这个粒度足够小，处理能力足够强；

文件属性分离到 FileStore 之后，单分片不需要处理占比最高的文件属性操作，压力远小于传统的设计。

6.2. 缩减分布式锁开销

6.2.1. 优化跨组件的协同开销

以 create、unlink、rename 为例，具体的执行过程如下：



文件系统在对文件操作之前都会经历从特定目录开始 lookup 的过程，只有 inode id record 存在对应的文件才会被看到，因此，只需要保证 file attributes record 比 inode id record 更早产生更晚消亡，就可以保证无效数据不会被用户看到，从外部视角观察，一致性没有被打破。图示给出的操作顺序可以达到这个效果，唯一的副作用是操作失败可能残留垃圾，这可以通过垃圾回收来解决。

这个方法可以推广到 TafDB 内部的情况，对于 inode id record 和 attribuets record 均存储在 TafDB 的目录也同样适用。当然，具体的操作执行顺序需要考虑 POSIX 的兼容性要求，可能和文件不完全一致，在此不做详细展开。

6.2.2. 优化 TafDB 内部的跨分片开销

在 TafDB 这样的系统中，如果一个事务涉及到多个分片，必然需要 2PC 事务提交机制来保证 ACID。我们采用的数据布局，让目录的 attribuets record 和其子项的 inode id record 都在一个分片内，使得这些必要的修改操作可以从传统的跨分片事务简化到单分片事务。在接下来的章节我们将介绍这个简化技术。经过这一次裁剪之后，CFS 中除了 Normal Path rename，所有的 TafDB 操作均是高度优化的单分片事务。

将元数据分散到 TafDB 和 FileStore 两个组件存储之后，首先要保证的是两个系统的对外一致性。我们通过精心排列的执行顺序来解决这个问题：

对于所有的创建操作，先创建 FileStore file attribuets record，后创建 TafDB inode id record；

对于所有的删除操作，先删除 TafDB inode id record，再删除 FileStore file attribue record；

就像xinghan先创建和删除文件在TafDB和Aries的流程

6.3. 缩减单分片锁

6.3.1. 单分片原语

关联变更涉及到多条记录，在执行过程中还存在 read-and-modify 模式，特别的，对父目录属性的修改就需要先读出旧值再更新。一个朴素的实现方式会包含多轮条件检查、读和写操作，效率肯定是不高的。为了解决这个问题，我们提出了单分片原语（single-shard atomic primitive）的概念。

每一种原语实现了一个定制的单分片事务，这个事务原子地完成所需要的所有读、写和条件检查，保证执行过程是 all-or-nothing 的，只有全部条件检查为真，才会执行成功，否则不会对分片数据做任何修改。原语作为一种特殊的单分片事务，高度优化后的执行代价和写一条 WAL 日志相当。

我们归纳了所有的 POSIX 操作，最后总结出三种原语，如下表所示，表中同时标注了每种原语的使用范围、类 SQL 形式的接口描述和简要的执行过程：

Primitives	Use cases	Syntax	Description
insert_ with_update	create mkdir symlink link	INSERT (value_list) WITH UPDATE table_name SET (assignment_list) WHERE (update_cond)	(1) check if the record that matches update_cond exists; (2) insert a new inode id record; (3) update the existing record according to assignment_list.
delete_ with_update	unlink rmdir	DELETE (delete_cond) WITH UPDATE table_name SET (assignment_list) WHERE (update_cond)	(1) check if records matching delete/update_cond exist; (2) delete the record that matches delete_cond; (3) update the record that matches update_cond according to assignment_list.
insert_ and_delete_ with_update	rename	INSERT (value_list) WITH DELETE (delete_cond_list) WITH UPDATE table_name SET (assignment_list) WHERE (update_cond)	(1) check if records that match their specified conditions exist; (2) delete records that match any condition in delete_cond_list; (3) insert a new inode id record with changes of primary key and fields specified by value_list; (4) update the record based on update_cond, assignment_list.

百度智能云技术站

论文里给了 create、unlink、同目录文件 rename 伪代码作为示例解释了原语是如何工作的。在这里我们可以看一下其中 create 的例子。

```
1  -- Insert file attribute into FileStore
2  FileStore.Put(key=@id, value={links=1, children=0, ...})
3  -- Insert inode record into TafDB while update parent
4  INSERT (kID=@parent_id, kStr=@name, id=@id)
5  WITH UPDATE inode_table
6  SET (children+=1, mtime=@now, ...)
7  WHERE (kID=@parent_id, kStr="/_ATTR", type=dir)
```

百度智能云技术站

create 的第一个步骤是在 FileStore 中创建文件，第二个步骤就是 insert_with_update 原语，主要完成了以下工作：

WHERE 做了两个条件检查，kID=@parent_id, kStr="/_ATTR", type=dir 这几个条件联合检查 kID、kStr、type 是否符合指定的值。当检查通过时意味着 inode 为 @parent_id 的 attribue record 在 TafDB 中存在，且类型为目录；

SET 对符合 WHERE 条件的记录更新属性，具体的就是更新 children、size、mtime 等字段；

INSERT 插入 inode id record 记录，主键是 kID=[@parent_id](#), kStr=[@name](#)，主要的字段是 inode；

在文章里原语是以类 SQL 的形式表达的，这只是为了表达的方便，实际 TafDB 提供的是 KV 层接口。相比于标准的接口，原语强化了条件检查的能力，同时在语句执行出错时能够返回更精细的出错条件，CFS 负责将这些条件转译成 POSIX 错误码。根据我们的经验，即使使用的 Table 系统本身不支持这样的扩展，进行二次开发所需的工作量也很小，和耦合式元数据服务相比，工作量和实现难度更是要低好几个数量级。

最后，我们总结一下原语的优势：

- 极大地降低了和 TafDB 的通信开销，交互轮次减到一次；
- 将很多的关联操作压缩到一次处理，降低了整体的处理损耗，并存在进一步优化的空间；
- 简化了元数据服务的设计，不同的 POSIX 操作可以被 3 种原语概括，传统的实现则需要实现所有的 POSIX 接口。

6.3.2. 冲突合并

如果沿用标准的实现，上述的原语仍然在父目录属性更新的时候导致排队。前文我们分析过，这里冲突的本质其实是对属性的更新操作，本质上是一些原子变量操作。根据这一点，我们进一步实现了两个增强机制用于弱化及合并处理事务冲突。

第一个机制是 **delta apply**，针对 links、children、size 这些数字属性。它们在变更的时候会加减增量，和原子变量加减一样，顺序并不重要，只要保证最终的叠加结果是对的。delta apply 就实现了这种合并加减效果。

第二个机制是 **last-writer-win**，针对权限、mtime、ctime 这些单纯的覆盖操作，我们简单的保留最后一个赋值操作的结果。

原语通过检查 SET 语句涉及的是加减运算（如 children+=1）还是赋值运算（如 mtime=[@now](#)）就可以自动决定运用 delta apply 还是 last-writer-win。这个检测完全不需要感知文件系统的语义，具有通用性，实际上可以推广到任何只是部分字段原子变量操作的场景，实现比写写冲突更小范围的冲突。

6.4. 移除元数据代理层

将 Normal Path rename 之外的所有操作均拆解成了原语的组合之后，这些操作之间只有落到单个分片上时才可能会产生冲突，分离式架构里的元数据代理层存在的唯一价值变成了接口翻译，串联起各个环节来实现 POSIX 接口，但这一点完全可以由客户端来承担，因此我们做了一个大胆的决定，将这一层功能完全集成到客户端来实现。

6.5. 强一致 Rename

rename 是一个文件系统里最复杂的操作，最坏的情况下，涉及到 6 个元数据对象，包括 4 个 attributes record，2 个 inode id records。根据我们的线上统计，99% 的 rename 都发生在同一个目录内文件间，这种 case 涉及到的变更都在一个 TafDB 分片内，可以采用前文提及的方法优化。因此，我们将 rename 分为 **Fast Path** 和 **Normal Path** 两种。

Fast Path 的 rename 基于 insert_and_delete_with_update 原语实现，只处理同一个目录内的文件 rename，剩下的其它类型均为 Normal Path。Normal Path 由 Renamer 进行处理，Renamer 是每个文件系统一个 Raft 复制组，负责对 Normal Path rename 操作进行冲突检测。只有通过冲突检测的请求会发往 TafDB 继续处理使用 2PC 事务进行处理，这个检测可以保证发出去请求不会导致孤儿或环。

Normal Path rename 和其它操作并发的正确性基于两个方面保证：

其它操作都不会改变子项的归属，顶多导致看到的信息不是最新的，但不会对成环或孤儿的判断产生实际影响。在处理 Normal Path rename 的 2PC 事务中，我们会对可能误判的情况进行进一步的检查，**如果出现变化则进行回退重试，整个处理过程和乐观锁机制类似**；

1PC 事务只是对普通 2PC 事务的高度优化，ACID 和事务隔离级别的保证没有被破坏，因此并发的 1PC、2PC 事务进一步由 TafDB 进行冲突处理，保证执行的结果符合隔离性和正确性承诺。

整个机制的正确性可以通过 TLA+ 之类的形式化验证手段证明。

6.6 垃圾回收

当 ClientLib 出现网络分区或者进程崩溃的时候，没有做完的操作会导致 TafDB 或 FileStore 中残留垃圾。系统通过两种机制进行处理这些垃圾：

周期对账：周期性地在 TafDB 和 FileStore 之间进行对账，回收垃圾数据；

按需处理：当 getattr、readdir 在执行的时候发现某些 attributes record 无法找到时可以马上发起一次垃圾回收，回收对应的 inode id record。

7. 实验

论文里将 CFS 和 HopsFS、InfiniFS 进行了详细的对比。测试结果显示，在 50 节点规模的测试中，与 HopsFS 和 InfiniFS 相比，CFS 各操作的吞吐量提高至 1.76 – 75.82 倍和 1.22 – 4.10 倍，并将它们的平均延迟分别最高降低了 91.71% 和 54.54%。在竞争较高和目录较大的情况下，CFS 的吞吐量优势则会进一步扩大一个数量级。

想了解完整测试结果的读者可以阅读论文原文的实验部分，在这里就不再赘述。

8. 总结

本文介绍了百度智能云文件存储 CFS 的元数据系统的核心设计，对长期困扰文件系统元数据领域的 POSIX 兼容性和高扩展性（特别是写扩展性）难以兼顾的问题，进行了解答。这是一个大规模分布式文件系统能否扩展到千亿级别文件数，同时保持高性能稳定性的一个关键问题。

分离式元数据架构是近年来文件系统元数据领域的发展趋势，业界有潜力存储千亿文件的系统均是基于这种架构实现的。这类架构采用类似“存算分离”的思想，**将元数据服务分为两层，分别是负责存储数据的数据库层，和偏计算逻辑、负责实现文件系统语义的元数据代理层**。但这种架构并没有解决写扩展性较差、写延时偏高的问题。

文件存储 CFS 进一步发展了分离式元数据架构，通过精心设计数据布局，让元数据的不同部分以更科学的方式在系统里打散和聚合。打散是为了提高数据处理的并行度，聚合则让相互依赖的数据避免多轮交互能够被一次就处理完。在这个数据布局的基础上，CFS 不断修剪元数据写操作临界区的范围，最终实现了无锁化，解决了分离式元数据架构写扩展性较差、写延时偏高的问题。

CFS 的这套设计已经在生产环境中稳定运行了超过 3 年时间，为云上蓬勃发展的的大数据、AI、容器、生命科学等场景的业务提供了有力支撑。

需要指出的是，CFS 的创新和整个百度沧海存储技术体系的大力支持是分不开的。合抱之木，生于毫末；九层之台，起于累土。正是因为前人和其它团队做了很多扎实的工作，我们才可以对一些不成熟、非常规的想法快速地进行验证和试错，并在验证完全之后迅速落地生产环境。在百度内部，这样的例子还有很多。

在研发 CFS 元数据系统的过程中，我们得到的另外一个收获是怀疑精神，技术上没有什么权威是不可挑战的，多问一句“从来如此便对吗”没什么坏处。

最后，再次感谢 CFS 论文的合作方中国科学技术大学先进数据系统实验室（Advanced Data Systems Laboratory, ADSL）的老师和同学们，这篇论文的面世离不开你们的共同努力。